

ÉTUDE DE DESIGN · MVP

Du POC au MVP :

Chatbot RAG Puls-Events

Rapport de gestion de projet

Data Engineer · Formation NLP & Bases vectorielles

Puls-Events · Août 2026

Responsable technique	Jérémy
Alternant	Dorra — Data Engineer en formation NLP
Date	Août 2026
Durée projet	12 semaines
Stack principale	GCP · Vertex AI · LangChain · ChromaDB · Gemini 1.5 Pro
Portfolio GitHub	https://github.com/dorrazch-hue/dorra-portfolio-data

01 Analyse et synthèse des besoins

1.1 Contexte et utilisateurs cibles

Puls-Events propose une plateforme de découverte d'événements culturels personnalisée selon la localisation, la période et les préférences de chaque utilisateur. Le POC réalisé il y a un mois a validé la faisabilité d'un moteur de recherche sémantique basé sur RAG. L'objectif est maintenant de passer à un MVP déployable en production, utilisable par de vrais utilisateurs.

Utilisateur	Profil	Besoin principal
Grand public	Utilisateurs de la plateforme Puls-Events	Trouver des événements pertinents via langage naturel
Équipe produit	Responsables Puls-Events	Valider la valeur ajoutée du chatbot sur les métriques d'engagement
Équipe tech	Développeurs internes	Intégrer le chatbot via API REST dans la plateforme existante

1.2 Besoins techniques identifiés

Après échange avec Jérémy (responsable technique), j'ai identifié quatre besoins concrets à adresser dans le MVP :

Mémoire conversationnelle	Retenir l'historique utilisateur pour personnaliser les échanges au fil du temps. Sans cette mémoire, chaque échange repart de zéro et le chatbot perd toute pertinence pour des utilisateurs réguliers.
Contexte géographique	Adapter dynamiquement les réponses à la localisation de l'utilisateur. Un événement à Paris n'est pas pertinent pour un utilisateur à Lyon.
Recherche web temps réel	Enrichir les réponses via un agent web (smolagents / Hugging Face) quand la base vectorielle ne contient pas l'information (événements très récents, données manquantes).
Monitoring de performance	Mesurer la qualité des réponses, la latence et la satisfaction utilisateur pour piloter les améliorations et détecter les dérives du modèle.

1.3 Cas d'usage concrets

#	Cas d'usage	Exemple
CU-01	Recherche d'événements par thème	« Quels concerts jazz ce weekend à Bordeaux ? »
CU-02	Recommandation personnalisée	« Donne-moi des idées d'événements pour ce soir »
CU-03	Suivi conversationnel	« Et la semaine prochaine ? » (référence au contexte précédent)
CU-04	Événement récent non indexé	L'agent smolagents cherche sur le web si la base vectorielle est vide

1.4 Contraintes fonctionnelles

- Latence maximale : réponse en moins de 3 secondes pour 95% des requêtes
- Disponibilité : 99% uptime en heures ouvrées (MVP non critique la nuit)
- Langue : français uniquement pour le MVP (multilingue hors périmètre)
- RGPD : historique de conversation conservé 30 jours maximum avec consentement explicite
- Intégration : le chatbot s'expose via API REST pour s'intégrer à la plateforme existante sans refonte front-end

1.5 Critères de réussite mesurables

Indicateur	Cible MVP	Outil de mesure
Taux de réponse pertinente	> 80%	Évaluations manuelles (sample 50 req/semaine)
Latence moyenne	< 2s	Langfuse (p50 et p95)
Taux d'erreur LLM	< 5%	Langfuse — traces en erreur
Satisfaction utilisateur (CSAT)	> 3,5/5	Widget de feedback intégré au chat
Couverture index vectoriel	> 90% des événements OpenAgenda	Script de vérification hebdomadaire

02 Analyse et choix de la solution cloud

Pour héberger le MVP, j'ai comparé les trois grands fournisseurs cloud selon des critères directement liés au projet : support IA/NLP, base vectorielle managée, facilité de déploiement et coût pour un volume de trafic MVP :

Critère	AWS	GCP	Azure
Services IA/NLP	Bedrock, SageMaker	Vertex AI, Gemini 1.5 Pro ✓	Azure OpenAI (OpenAI)
Base vectorielle managée	OpenSearch Serverless	Vertex AI Vector Search ✓	Azure AI Search
Facilité déploiement	★★★ (complexe IAM)	★★★★ ✓	★★★ (couplé Azure AD)
Intégration Hugging Face	Limitée	Native via Vertex ✓	Partielle
Coût MVP (estimation)	~250 €/mois	~210 €/mois ✓	~280 €/mois
Écosystème open source	Bon	Excellent ✓	Moyen

Mon choix : GCP (Google Cloud Platform). Vertex AI couvre à la fois les embeddings (text-embedding-gecko-003, région us-central1) et le LLM (Gemini 1.5 Pro) dans un seul fournisseur. L'intégration avec Hugging Face facilite l'utilisation de smolagents. Cloud Run gère le scaling automatiquement — idéal pour un MVP à charge incertaine. Le coût est aussi le plus compétitif des trois pour notre volumétrie cible.

03 Architecture technique détaillée du MVP

Rappel : le POC existant (P11) repose sur **Mistral AI + FAISS** — il a validé la faisabilité. Le MVP décrit ici constitue la **migration vers GCP** : embeddings Vertex AI, ChromaDB puis Vector Search, Gemini 1.5 Pro. L'architecture MVP s'appuie sur les enseignements du POC en ajoutant trois couches : mémoire conversationnelle, géolocalisation et agent de recherche web. Elle se décompose en 4 niveaux :

1. Interface & API

FastAPI exposé sur Cloud Run (conteneur Docker). Endpoint POST /chat avec authentification par token Bearer. La documentation Swagger est auto-générée. Cloud Run gère le TLS, les certificats et l'autoscaling.

2. Moteur RAG

Embeddings : text-embedding-gecko-003 (Vertex AI, us-central1)

Base vectorielle : ChromaDB en phase MVP (voir note ci-dessous) → Vertex AI Vector Search en production

LLM : Gemini 1.5 Pro (Vertex AI)

Orchestration : LangChain — gère le prompt, le contexte et les appels outils

3. Modules complémentaires

Mémoire : historique de session stocké dans Firestore (document par user_id), fenêtre de 10 échanges

Géolocalisation : filtre metadata appliqué avant la recherche vectorielle (ville, département)

Recherche temps réel : agent smolagents déclenché si score RAG < 0.6 ou événement < 7 jours

4. Monitoring & Observabilité

Langfuse (self-hosted sur Cloud Run) : traçabilité complète des appels LLM, latence, tokens, erreurs

Cloud Logging : logs applicatifs et d'infrastructure

Métriques custom : CSAT (widget feedback), taux de pertinence, couverture index

Flux d'une requête (de bout en bout) :

```
1. Utilisateur → POST /chat (FastAPI · Cloud Run) 2. Récupération contexte → Firestore (historique) + Géoloc (ville) 3. Embedding requête → text-embedding-gecko (Vertex AI) 4. Recherche vectorielle → ChromaDB (filtré par géoloc + score > 0.6) 5a. Score OK → Prompt LLM → Gemini 1.5 Pro (Vertex AI) → Réponse 5b. Score < 0.6 → Agent smolagents → recherche web → Prompt LLM → Réponse 6. Mise à jour Firestore (historique) + Trace Langfuse
```

Note importante — ChromaDB et persistance partagée

ChromaDB embarqué dans un conteneur Cloud Run n'est pas persistant entre les instances : si Cloud Run démarre plusieurs instances en parallèle (scalabilité horizontale), chaque instance aurait sa propre copie de l'index — ce qui est incohérent. **Solution retenue pour le MVP** : ChromaDB en mode client-serveur, déployé sur une instance Cloud Run dédiée avec un volume Cloud Storage FUSE monté pour la persistance. En production, on migrera vers **Vertex AI Vector Search** (managé, répliqué, faible latence).

Stratégie de scalabilité

Paramètre	Configuration MVP	Évolution production
Instances Cloud Run	min=1, max=5, concurrency=10	min=2, max=20, concurrency=20
Mise en cache	Cache LRU 15 min sur requêtes fréquentes (LangChain cache)	Redis Cloud (Memorystore GCP)
Index vectoriel	ChromaDB HNSW (ANN, ef=100)	Vertex AI Vector Search (ScaNN, SLA 10ms p99)
Reconstruction index	Nightly batch (Cloud Scheduler)	Streaming updates via Pub/Sub
Gestion indisponibilité	Health check Cloud Run, restart auto	Multi-région, circuit breaker LangChain

04 Macro backlog

Les fonctionnalités sont priorisées avec le modèle MoSCoW. Les estimations d'effort sont exprimées en jours/homme (alternant). Les dépendances indiquent les items qui doivent être terminés avant.

Priorité	Epic	Fonctionnalité	Effort	Dépendance
Must	RAG Core	Indexation événements Open Agenda	3j	—
Must	RAG Core	Recherche sémantique vectorielle	4j	Indexation
Must	RAG Core	Génération de réponses via Gemini 1.5 Pro	3j	Recherche vectorielle
Must	API	Endpoint REST FastAPI + Swagger	2j	RAG Core
Must	Mémoire	Historique de conversation (Firestore)	3j	API
Must	Géoloc	Filtrage résultats par localisation	2j	Indexation
Should	Recherche	Intégration agent smolagents	4j	RAG Core
Should	Monitoring	Dashboard Langfuse (latence, qualité)	3j	API
Should	Sécurité	Auth token + gestion secrets GCP	2j	API
Could	UX	Interface chat intégrée Puls-Events	5j	API
Could	Monitoring	Score CSAT utilisateur	2j	Langfuse
Won't	Multilingue	Support langues autres que FR	—	Hors MVP

Effort total estimé Must : ~17 jours/homme. Should : ~9 jours. Total MVP complet (Must + Should) : ~26 jours sur 12 semaines en alternance (environ 3 jours/semaine en entreprise).

05 **Registre des risques**

J'ai identifié les principaux risques du projet et leur impact sur le planning. Probabilité et impact sont notés sur 3 (1=faible, 2=moyen, 3=élevé). Les risques de criticité ≥ 6 sont intégrés comme buffers dans le planning.

#	Risque	Prob.	Impact	Criticité	Mitigation
R1	Latence Gemini 1.5 Pro > 3s en pic de charge	2	3	6 — Élevé	Cache LRU + fallback Gemini 1.5 Flash (moins cher, plus rapide)
R2	Qualité RAG insuffisante (hallucinations)	2	3	6 — Élevé	Évaluation hebdomadaire sur 50 requêtes + ajustement prompt + reranking
R3	ChromaDB : perte d'index au redémarrage Cloud Run	3	2	6 — Élevé	Déploiement ChromaDB avec volume persistant (GCS FUSE) + backup quotidien
R4	API smolagents instable ou quota dépassé	2	2	4 — Moyen	Mode dégradé sans recherche web + monitoring quota
R5	Coût Vertex AI dépassant le budget	2	2	4 — Moyen	Budget alert GCP (seuil 80%) + limite tokens/requête + cache
R6	Données OpenAgenda incomplètes ou incorrectes	2	2	4 — Moyen	Script de validation qualité avant indexation
R7	Retard planning (charge alternance)	2	2	4 — Moyen	Buffer de 1 semaine en S11 (Tests) + priorisation Must uniquement si retard
R8	Violation RGPD (données de localisation)	1	3	3 — Faible	Voir section Sécurité & RGPD — consentement explicite + anonymisation

Le chatbot collecte et traite deux catégories de données personnelles sensibles : la géolocalisation et l'historique de conversation. Ces données sont encadrées par le RGPD et nécessitent une gestion explicite.

6.1 Données collectées et base légale

Donnée	Finalité	Base légale	Durée de conservation
Géolocalisation (ville)	Filtrage des événements par zone	Consentement (opt-in)	Session uniquement
Historique de conversation	Mémoire conversationnelle	Consentement (opt-in)	30 jours maximum
Logs d'utilisation (anonymisés)	Monitoring et amélioration	Intérêt légitime	90 jours

6.2 Mesures techniques de protection

- **Consentement explicite** : Un bandeau de consentement s'affiche avant le premier échange, expliquant clairement quelles données sont collectées et pourquoi. L'utilisateur peut refuser sans perdre l'accès au chatbot (la géoloc et la mémoire sont alors désactivées).
- **Anonymisation des logs** : Les requêtes Langfuse sont loguées sans identifiant utilisateur direct (hash SHA-256 de l'user_id). Le contenu des conversations n'est pas archivé dans les logs Langfuse.
- **Contrôle d'accès** : Authentification par token Bearer (JWT signé). Les secrets (clés API Vertex, tokens Langfuse) sont stockés dans Google Secret Manager, jamais en variables d'environnement en clair.
- **Chiffrement** : Toutes les communications sont chiffrées TLS 1.3 (Cloud Run gère automatiquement). Les données Firestore sont chiffrées au repos par Google (AES-256).
- **Droit à l'oubli** : Un endpoint DELETE /user/{id}/history permet de supprimer l'historique d'un utilisateur à sa demande, conformément à l'article 17 du RGPD.

07 Estimation des coûts

7.1 Build — Développement et déploiement initial

Poste	Détail	Coût estimé
Développement alternant	3 mois × coût entreprise (~1 000 €/mois)	~3 000 €
Infrastructure setup GCP	Projet GCP, CI/CD (Cloud Build), environnements dev/staging/prod	~150 €
Vertex AI — phase dev	Tests d'embeddings et LLM en développement (usage limité)	~50 €
Total build		~3 200 €

7.2 OPEX — Exploitation mensuelle (hypothèse : 1 000 utilisateurs actifs, 50 requêtes/utilisateur/mois soit 50 000 requêtes/mois — modèle Gemini 1.5 Pro, région us-central1)

Service	Usage détaillé	Tarif unitaire	Coût/mois
Vertex AI — Gemini 1.5 Pro	50K req × 800 tokens input + 400 tokens output	0,00125 \$/1K tokens input 0,005 \$/1K tokens output	~50 + 100 = ~150 €
Vertex AI — Embeddings (text-embedding-gecko-003)	50K req × 500 tokens	0,0001 \$/1K tokens	~3 €
Cloud Run — API	Serverless, faible charge MVP (< 100 req/h en moyenne)	Premiers 2M req gratuits/mois	~15 €
Cloud Run — ChromaDB	Instance dédiée 1 vCPU / 2 GB RAM, 24/7	~0,024 \$/vCPU/h	~18 €
Firestore — Mémoire	50K lectures + 50K écritures, < 1 GB stockage	0,06 \$/100K lectures	~3 €
Langfuse (self-hosted)	Cloud Run 0,5 vCPU + PostgreSQL Cloud SQL minimal	Inclus dans Cloud Run	~10 €
Smolagents / Recherche web	~10% des req déclenchent la recherche web (5K appels)	Serper API 0,001 \$/req	~5 €
Cloud Logging + Storage	Logs, backups index vectoriel, GCS FUSE	0,02 \$/GB/mois	~4 €
Total OPEX			~210 €/mois

Hypothèse de montée en charge : à 10 000 utilisateurs actifs (×10), l'OPEX monterait à environ 1 700-2 000 €/mois, essentiellement piloté par le coût Gemini 1.5 Pro (~1 500 €). À ce stade, migration vers Vertex AI Vector Search recommandée (+~80 €/mois mais gain de performance significatif) et activation de Gemini 1.5 Flash pour les requêtes simples (-60 à -70% sur le coût LLM).

7.3 Optimisation budgétaire

Levier	Description	Économie estimée
Cache de requêtes	LangChain cache en mémoire (15 min) pour les requêtes identiques	-15 à -20% coût LLM
Fallback modèle	Gemini 1.5 Flash pour les requêtes simples (10× moins cher)	-30% coût LLM si 40% des req.
Limite tokens	Max 600 tokens input / 300 output par requête via prompt engineering	-20% coût LLM
Budget alert GCP	Alerte à 80% du budget mensuel, coupure à 110%	Évite dépassement
Batching indexation	Mise à jour de l'index en batch nocturne (vs. temps réel)	-40% coût embeddings

08 **Plan de projet**

Le projet est découpé en 6 phases sur **12 semaines** avec 4 jalons de validation formels avec Jérémy. Les risques critiques (R1, R2, R3) sont intégrés au planning avec des buffers dédiés.

Phase	Sem.	Activités principales	Livrable	Risques couverts
1. Cadrage & Setup	S1–S2	Architecture validée, setup GCP, environnements dev/staging	Doc technique validée par Jérémy	R3 (ChromaDB persistance)
2. RAG Core	S3–S5	Indexation OpenAgenda, recherche vectorielle, Gemini 1.5 Pro	Chatbot v1 fonctionnel	R2 (qualité RAG) R6 (données)
3. Enrichissement	S6–S8	Mémoire Firestore, géoloc, agent smolagents	Chatbot v2 complet	R4 (smolagents)
4. Monitoring & Sécurité	S9–S10	Langfuse, métriques CSAT, auth JWT, secrets GCP	Monitoring actif + RGPD conforme	R5 (coûts) R8 (RGPD)
5. Tests & Optimisation	S11	Tests de charge, corrections, cache LRU, optimisation prompt	Rapport de tests + optimisations	R1 (latence) Buffer risques
6. Livraison MVP	S12	Déploiement prod, documentation technique, formation équipe	MVP en production + documentation	R7 (planning)

Jalons clés

Jalon	Critère de validation
J1 — S2	Architecture validée par Jérémy + environnement de développement opérationnel
J2 — S5	Démo RAG Core : chatbot de base fonctionnel sur données réelles
J3 — S8	Démo MVP complet : toutes fonctionnalités (mémoire, géoloc, smolagents)
J4 — S12	Livraison production + documentation + monitoring opérationnel

Bilan POC → MVP : étapes, arbitrages et recommandations

Cette section retrace le chemin parcouru depuis le POC jusqu'à la définition du MVP, les choix effectués, les difficultés rencontrées et les décisions encore ouvertes.

9.1 Ce que le POC a apporté

- Validation de la faisabilité d'un moteur RAG sur des données événementielles réelles
- Validation du modèle d'embeddings Mistral (mistral-embed, 1 024 dimensions) sur données événementielles françaises
- Mesure des premiers temps de réponse : ~4-6s sans optimisation — trop lent pour la prod
- Conviction des équipes produit et marketing sur la valeur du chatbot

9.2 Principales évolutions POC → MVP

Dimension	POC	MVP
Architecture	Script local, pas de serveur	API Cloud Run containerisée
Embeddings	Mistral-embed (1 024 dims, via API)	text-embedding-gecko-003 (Vertex AI, native GCP)
Base vectorielle	FAISS (IndexFlatL2, persisté en pickle)	ChromaDB persistant (GCS) → Vertex AI Vector Search
LLM	Mistral AI mistral-small (appels directs)	Gemini 1.5 Pro via LangChain (cache, retry, fallback)
Mémoire	Aucune (stateless)	Firestore — historique 10 échanges par session
Monitoring	Aucun	Langfuse complet + métriques CSAT
Sécurité	Pas d'auth	JWT Bearer + Secret Manager GCP
Tests	Tests manuels ponctuels	Suite de tests automatisés + tests de charge S11

9.3 Arbitrages effectués

- **ChromaDB vs. Vertex AI Vector Search** : j'ai choisi de démarrer avec ChromaDB pour aller vite et limiter les coûts. La migration vers Vertex AI Vector Search est planifiée pour la phase de production dès que la charge justifie l'investissement (~80 €/mois supplémentaires).
- **Gemini 1.5 Pro vs. GPT-4** : GCP offre un écosystème intégré (Vertex AI, embeddings, Cloud Run) qui simplifie la gestion et réduit les appels réseau inter-services. GPT-4 aurait nécessité une dépendance externe (OpenAI) avec des coûts plus élevés.
- **Interface chat hors scope MVP** : L'équipe produit souhaitait une interface intégrée. J'ai proposé de la reporter en Could pour prioriser la solidité de l'API et du RAG — la valeur est dans le moteur, pas dans l'interface pour un MVP interne.

9.4 Difficultés rencontrées et solutions

Difficulté	Solution apportée
Persistance ChromaDB sur Cloud Run (instances stateless)	Architecture client-serveur avec volume GCS FUSE monté
Latence RAG > 3s au POC	Cache LRU LangChain + optimisation du prompt (réduction tokens input)
Données OpenAgenda incomplètes	Script de validation et nettoyage avant indexation + fallback smolagents
Complexité RGPD géolocalisation	Collecte uniquement de la ville (pas de coordonnées GPS exactes) + consentement opt-in

9.5 Recommandations et prochaines décisions

- Valider le choix ChromaDB vs. Vertex AI Vector Search avec Jérémie à J2 (S5) selon la charge observée
- Mettre en place dès S1 un budget alert GCP pour éviter les surprises de coût
- Planifier une revue de sécurité RGPD avec le DPO de Puls-Events avant le déploiement prod (J4)
- Envisager un A/B test Gemini 1.5 Pro vs. Gemini 1.5 Flash sur 20% du trafic pour optimiser les coûts
- Documenter l'API (Swagger) dès S2 pour faciliter l'intégration par l'équipe front-end

A. Portfolio professionnel

Le portfolio GitHub regroupe les projets réalisés pendant le parcours Data Engineer (NLP, bases de données vectorielles, gestion de projet). Il comprend pour chaque projet le contexte, les outils utilisés, les résultats obtenus et les compétences démontrées.

■ Portfolio GitHub : <https://github.com/dorrazch-hue/dorra-portfolio-data> Projet principal : Chatbot RAG Puls-Events — Du POC au MVP (ce projet)

B. Glossaire

Terme	Définition
RAG	Retrieval-Augmented Generation — technique combinant recherche vectorielle et génération LLM
LLM	Large Language Model — modèle de langage (ici : Gemini 1.5 Pro)
Embeddings	Représentations vectorielles du texte pour la recherche sémantique
ChromaDB	Base de données vectorielle open source utilisée pour stocker et interroger les embeddings
LangChain	Framework Python d'orchestration des composants LLM (prompt, mémoire, outils)
smolagents	Bibliothèque Hugging Face pour créer des agents IA capables de rechercher sur le web
Langfuse	Outil de monitoring et d'observabilité pour les applications LLM
Cloud Run	Service GCP de déploiement de conteneurs serverless à auto-scaling
Vertex AI	Plateforme IA/ML de Google Cloud (embeddings, LLM, Vector Search)
MoSCoW	Méthode de priorisation : Must / Should / Could / Won't have
CSAT	Customer Satisfaction Score — indicateur de satisfaction utilisateur (1 à 5)
ANN	Approximate Nearest Neighbor — algorithme de recherche vectorielle rapide (HNSW, ScaNN)